

pandas 시험 예상 정리 (1~5장, 이해 위주)

교수님 수업 멘트 + 노트북 15개(2~5장) + python_101 기준으로 추측한 것. 2026-10-02 작성

읽기 전에: 이건 어디까지나 예측이에요. 교수님이 "중요하다"고 직접 말한 것은 ★★★, 수업에서 강조했다고 필기한 것은 ★★, 힌트 없이 흐름상 나올 법한 것은 ★로 표시했어요. 4장 뒷부분(2~4번 노트북)과 5장은 수업 진도가 아직이라 시험 범위에 들어가는지 교수님께 확인하는 게 먼저예요.

1. 교수님이 수업에서 직접 주신 힌트

어디서	교수님 멘트 / 필기	해석
3-1 wide vs long	"wide, long이 제일 중요한 듯"	wide/long 판별 + 서로 바꾸는 법 + 장단점은 거의 확정
3-3 cleaning	rename, df.columns, inplace 설명 / to_datetime, astype 강조 / 정렬 과정 중요 / nlargest, reset_index	타입 변환과 정렬은 "코드 → 결과" 또는 "결과 → 코드 빈칸" 형태로
3-3 비트코인+S&P;	"groupby 중요함" / 날짜 같으면 합쳐주는 방식	groupby(level='date').sum() 과 왜 주말 값이 이상한지(reindex)
3-4 reshaping	"melt 시험문제 내긴 함" / "pivot 중요함" / unstack은 "이런 방법도 있다"	melt, pivot은 코드 쓰기까지 가능하게. unstack은 개념만
3-5 data issues	np.inf 언급	무한대/NaN 찾는 법 정도. 비중 낮음
4-1 querying	"SQL 안 냄" / "Boolean mask 추천"	query() 대신 df[(조건) & (조건)] 형태로 쓸 줄 알아야 함
4-1 merging	수업에서 다룸	inner / left / right / outer 차이 (행 수, NaN 위치)

2. 장별 출제 예상 포인트

1장 (python_101) — 2장 문제 속에 녹아서 나올 가능성

별점	포인트	왜 / 어떻게 나올까
★★	슬라이싱 끝 번호 미포함 my_list[1:3]	df[100:103], iloc의 뿌리. "몇 개 나오나?" 형태
★★	= (대입) vs == (비교), and/or	pandas에서는 and/or 대신 &, 써야 하는 것과 대비
★	리스트 vs 튜플(수정 불가), 딕셔너리 {키: 값}	튜플 수정 에러 이유 / 딕셔너리로 DataFrame 만들기
★	리스트 컴프리헨션 [x for x in ... if 조건]	2장 열 고르기, 4장 열 이름 합치기에 계속 등장

2장 (pandas 기초, 노트북 6개) — 직접 힌트는 없지만 3~4장의 뿌리

별점	포인트	왜 / 어떻게 나올까
★★★	Boolean mask 필터링: 조건마다 괄호 + & /	교수님이 4장에서 "마스크 추천". 원조가 5_subsetting. "틀린 곳 찾기" 단골
★★★	인덱스 맞춰서 더하기 (x + y 에서 NaN)	"왜 NaN이 생기나?" 전형적인 이해 문제. 3장 비트코인 예제의 원리
★★★	loc(이름, 끝 포함) vs iloc(번호, 끝 미포함)	같은 10:15인데 6행 vs 5행. 거의 모든 pandas 시험의 단골
★★	concat: axis=0 위아래 / axis=1 좌우, 인덱스 안 맞으면 NaN + 행 증가	3장 비트코인 concat으로 이어짐
★★	assign + lambda x: 방금 만든 열을 바로 쓰기	3장 temp_F_whole, 4장 곳곳에서 반복. "왜 lambda가 필요한가"

별점	포인트	왜 / 어떻게 나올까
★★	del / pop / drop 차이, drop은 원본 안 바꿈	inplace 개념으로 연결
★	shape / head / info(Non-Null Count) / describe(25·50·75%) / value_counts vs unique	결과 읽기 문제
★	dtypes: object = 문자열, float64, int64	3장 타입 변환의 전제
낮음	numpy 앞부분, %%timeit, !터미널 명령어, sqlite, API(requests), 정규표현식 체이닝	실습 배경이라 이해 문제로 내기 애매함

3장 (데이터 정리, 노트북 4개) — 힌트가 가장 많음 = 가장 많이 나올 곳

별점	포인트	왜 / 어떻게 나올까
★★★★	wide vs long 판별 + 각각의 장단점	"제일 중요" 멘트. 표 보여주고 "wide인가 long인가, 왜?"
★★★★	melt() (wide→long), pivot() (long→wide)	"melt 시험 낸다", "pivot 중요". 옵션(id_vars, value_vars, index, columns, values) 빈칸 채우기
★★★★	sort_values(by=[a, b], ascending=[False, True])	"정렬 과정 중요". TMAX만 골라서 2개 기준 정렬하는 코드 쓰기
★★★★	groupby로 날짜 같은 행 합치기 + 비트코인/S&P; 주말 문제	"groupby 중요". 왜 주말에 값이 떨어지나 → reindex + ffill
★★	pd.to_datetime(), astype('int'), astype('category'), dtypes로 확인	"강조하심". 왜 날짜 타입이어야 하나 (날짜 슬라이싱, 월별 묶기)
★★	rename(columns={...}) + inplace=True 의미	df = df.rename(..., inplace=True) 함정 (None이 됨)
★★	nlargest / nsmallest, reset_index / ignore_index=True	sort 대신 쓰는 법, 정렬 후 인덱스 다시 매기기
★★	set_index('date') 후 날짜 문자열로 자르기 df['2018-10-11':'2018-10-12']	왜 set_index를 하는지
★	unstack / stack (pivot, melt의 MultiIndex 버전)	"이런 방법도 있다" → 개념만
★	NaN 찾기는 isna() (== np.nan 은 안 됨), np.isfinite	3-5는 가볍게. duplicated / dropna / fillna 이름 정도
낮음	tz_localize / tz_convert, to_period, 그래프 꾸미기 코드	수업에서 안 다룸

4장 (합치기·집계, 노트북 4개) — 1번만 진도 나감

별점	포인트	왜 / 어떻게 나올까
★★★★	merge 4종류: inner / left / right / outer	행 개수 전후 비교, NaN이 어느 열에 생기나, left_on/right_on vs on
★★★★	Boolean mask로 필터링 (query 대신)	"마스크 추천, SQL 안 냄". query 코드 주고 마스크로 바꾸기
★★	groupby('열').mean() / ['열'].agg([...])	"groupby 중요" (3장에서 이미 언급). 작은 표로 결과 예측
★ (진도 나가면)	agg vs transform (행 수 유지), pivot_table, crosstab	3장 pivot과 연결. transform은 "비율 구할 때"
★ (진도 나가면)	rolling(최근 N일) / expanding(누적) 개념, resample('1D'), shift / diff	시계열은 개념 위주로
낮음	SQL, pipe, window_calc, VariableOffsetWindowIndexer, merge_asof 세부 옵션	"SQL 안 냄" + 너무 세부적

5장 (그래프, 노트북 3개) — 힌트 없음, 범위 확인 필요

별점	포인트	왜 / 어떻게 나올까
★★	상황 → 그래프 종류 고르기 (line / scatter / hist-box / bar)	이해 위주 교수님이 제일 내기 좋은 형태. 코드 없이도 낼 수 있음
★★	Figure(도화지) vs Axes(칸), plt.subplots(1, 3) → fig, axes, ax=axes[0]	1번 노트북의 존재 이유
★	alpha(겹침), logx(값 차이 큼), stacked(합+비중), subplots=True + layout	"왜 이 옵션을 썼나" 형태
★	groupby / pivot_table로 정리한 뒤 .plot() 붙이는 흐름	4장 복습 겸
낮음	rcParams, gridspec, 히트맵·ECDF 꾸미기 코드, pandas.plotting 모듈 세부	참고용. lag plot은 "추가는 어제와 비슷하다" 개념만

3. 예상 문제 20개 (정답은 4번 섹션에)

이해 위주 시험에서 자주 나오는 5가지 형태로 만들었어요: (1) 코드 → 결과 예측 (2) 결과 → 코드 빈칸 (3) 틀린 곳 찾기 (4) "왜 그런가?" (5) 상황 → 방법 고르기. 모든 문제에서 df 인덱스는 0, 1, 2, ... 기본값이라고 가정.

Q1. [2장] 다음 코드의 결과를 쓰시오. 그리고 왜 그런 결과가 나오는지 한 줄로 설명하십시오.

```
x = pd.Series([10, 20, 30], index=[0, 1, 2])
y = pd.Series([1, 2, 3], index=[1, 2, 3])
x + y
```

Q2. [2장] 두 코드가 돌려주는 행 개수는 각각 몇 개인가? 차이가 나는 이유는?

```
df.loc[10:15, 'mag']
df.iloc[10:15, 8]
```

Q3. [2장] 아래 코드는 에러가 난다. 틀린 곳 두 군데를 고치고, 각각 왜 틀렸는지 설명하십시오.

```
df[df.mag > 5 and df.tsunami == 1]
```

Q4. [2장] df가 (9332, 7) 크기일 때 아래 코드를 실행하면 df.shape는? 'USGS'는 몇 행에 들어가는가? 이 현상을 뭐라고 부르나?

```
df['source'] = 'USGS'
```

Q5. [2장] a는 인덱스가 0, 1인 2행짜리 표, b는 인덱스가 100, 101인 2행짜리 표이다. 결과의 행 개수와 NaN이 생기는 이유를 쓰시오.

```
pd.concat([a, b], axis=1)
```

Q6. [2장] 아래 코드 실행 후 df.columns에 'mag'가 남아있는가? 그 이유와, 실제로 지우려면 어떻게 써야 하는지 두 가지 방법을 쓰시오.

```
df.drop(columns='mag')
df.columns
```

Q7. [2장] 두 번째 열(neither)만 lambda를 쓴 이유를 설명하십시오. lambda 없이 ~df.in_ca 라고 쓰면 어떻게 되나?

```
df.assign(
    in_ca=df.place == 'California',
    neither=lambda x: ~x.in_ca
)
```

Q8. [3장] 아래 표는 wide인가 long인가? 판단 근거를 쓰고, 반대 형식으로 바꾸는 코드를 작성하십시오. (열: date, TMAX, TMIN, TOBS)

	date	TMAX	TMIN	TOBS
0	2018-10-01	21.1	8.9	13.9
1	2018-10-02	23.9	13.9	17.2

Q9. [3장] wide 형식과 long 형식의 장점을 각각 한 가지 이상 쓰시오. long 형식 데이터에 describe()를 하면 결과가 왜 의미 없는가?

Q10. [3장] "datatype이 TMAX인 행만 골라서, temp_C는 내림차순, temp_C가 같으면 date는 오름차순으로 정렬하고 상위 10개" 를 코드로 쓰시오. 정렬 없이 큰 순서로 10개만 뽑는 다른 방법도 쓰시오.

Q11. [3장] 아래 코드를 실행하면 마지막 줄에서 에러가 난다. 이유와 고치는 법(두 가지)을 쓰시오.

```
df = df.rename(columns={'value': 'temp_C'}, inplace=True)
df.head()
```

Q12. [3장] date 열이 object(문자열)이고 temp_C가 float64이다. (a) date를 날짜 타입으로, (b) temp_C의 정수 부분만 가진 temp_C_whole 열을 만드는 코드를 쓰고, (c) 바뀐 걸 확인하는 명령어와 (d) date를 굳이 날짜 타입으로 바꿔야 하는 이유를 쓰시오.

Q13. [3장] S&P; 500(주말 데이터 없음)과 비트코인(매일 데이터)을 날짜별로 합쳐 포트폴리오 가치를 구했더니 주말마다 값이 뚝 떨어진다. (a) 왜 그런가? (b) 어떻게 고치는가? (코드 포함)

```
portfolio = pd.concat([sp, bitcoin]).groupby(level='date').sum()
```

Q14. [3장] 아래 코드는 0행을 돌려준다. 이유와 올바른 코드를 쓰시오.

```
df[df.TMAX == np.nan]
```

Q15. [4장] station_info는 관측소 279개(관측소당 1행), weather는 78,780행(관측소 110개 등장, 전부 station_info에 존재)이다. 다음 각 merge 결과의 행 개수를 쓰고, right/outer에서 NaN이 생기는 열은 어느 쪽 표의 열인지 쓰시오.

```
weather.merge(station_info, left_on='station', right_on='id') # (a) inner
weather.merge(station_info, left_on='station', right_on='id', how='right') # (b)
weather.merge(station_info, left_on='station', right_on='id', how='outer') # (c)
```

Q16. [4장] 아래 query 코드를 Boolean mask 방식으로 바꿔 쓰시오.

```
weather.query('datatype == "SNOW" and value > 0 and station.str.contains("US1NY")')
```

Q17. [4장] 아래 표에서 두 코드의 결과(값과 행 개수)를 쓰시오. agg(mean)와 transform(mean)의 차이는?

```
city temp
0 Seoul 10
1 Busan 20
2 Seoul 30

df.groupby('city').temp.mean()
df.groupby('city').temp.transform('mean')
```

Q18. [4장] 분 단위 주가를 하루 단위로 뭉치려 한다. 빈칸 (1)~(5)에 들어갈 집계 방법을 쓰고 이유를 설명하시오.

```
stock_per_minute.resample('1D').agg({
    'open': (1), 'high': (2), 'low': (3), 'close': (4), 'volume': (5)
})
```

Q19. [5장] 다음 상황에 맞는 그래프 종류(kind)를 고르고 이유를 쓰시오. (a) 1년간 주가가 어떻게 변했나 (b) 거래량과 가격 변동폭의 관계 (c) 지진 규모가 어떤 값에 몰려 있나 (d) 지역별 지진 횟수 비교 (e) 나라별 확진자의 합계와 각 나라 비중을 동시에

Q20. [5장] 아래 코드에서 fig와 axes는 각각 무엇인가? 두 번째 줄은 어디에 무엇을 그리는가? alpha=0.4를 준 이유는?

```
fig, axes = plt.subplots(1, 3, figsize=(15, 5))
df[['Italy', 'Spain']].plot(ax=axes[1], alpha=0.4)
```

4. 정답 및 해설

A1.

0: NaN, 1: 21, 2: 32, 3: NaN. pandas는 "위치"가 아니라 "인덱스 이름표"가 같은 것끼리 더함. 인덱스 0은 x에만, 3은 y에만 있어서 짝이 없음 → NaN.

A2.

loc는 6행(10~15, 끝 포함), iloc는 5행(10~14, 끝 미포함). l = label(이름)이라 "15번이라는 이름까지", i = integer(위치)라 파이썬 슬라이싱처럼 끝 전까지.

A3.

(1) and → & (pandas는 행마다 True/False를 비교해야 하는데 and는 값 하나짜리용이라 에러) (2) 조건마다 괄호 (& 가 > 보다 먼저 계산되기 때문).

```
df[(df.mag > 5) & (df.tsunami == 1)]
```

A4.

shape는 (9332, 8). 모든 행(9332행)에 'USGS'가 들어감. 값 하나를 모든 행에 퍼뜨리는 걸 broadcast(브로드캐스트)라고 함.

A5.

4행. axis=1(좌우)로 붙일 때는 인덱스가 같은 행끼리 옆에 붙는데, 0·1과 100·101은 하나도 안 겹쳐서 각자 따로 행이 생기고 짝이 없는 쪽은 NaN. 옆으로 붙일 땐 인덱스가 맞는지 먼저 확인!

A6.

남아있음. drop은 원본을 안 바꾸고 "지워진 새 표"를 돌려주는데 그걸 아무 데도 저장 안 했기 때문. 방법 1: df = df.drop(columns='mag') / 방법 2: df.drop(columns='mag', inplace=True). (del df['mag'] 도 가능)

A7.

in_ca는 지금 만들어지는 중이라 df에는 아직 없음. lambda x: 의 x에는 "지금 만들어지고 있는 표"가 들어오므로 x.in_ca로 쓸 수 있음. ~df.in_ca라고 쓰면 df에 in_ca 열이 없어서 AttributeError.

A8.

wide. 날짜 하나가 한 행이고 측정 항목(TMAX, TMIN, TOBS)이 각각 열로 펼쳐져 있기 때문. long으로 바꾸려면 melt:

```
wide_df.melt(
    id_vars='date',          # 그대로 둘 열
    value_vars=['TMAX', 'TMIN', 'TOBS'],  # 녹일 열들
    var_name='datatype', value_name='temp_C'  # 열 이름 → datatype, 값 → temp_C
)
# 반대(long → wide)는 pivot:
long_df.pivot(index='date', columns='datatype', values='temp_C')
```

A9.

wide 장점: 사람이 보기 편함, 열끼리 계산·describe·pandas plot이 쉬움. long 장점: 항목이 늘어도 행만 추가되어 구조가 안 바뀜, groupby/seaborn에 잘 맞음, DB 저장에 좋음. long의 value 열에는 TMAX·TMIN·TOBS 값이 다 섞여 있어서 평균·최소·최대를 내도 "무엇의" 통계인지 알 수 없음.

A10.

```
df[df.datatype == 'TMAX'].sort_values(
    by=['temp_C', 'date'], ascending=[False, True]
).head(10)
# 정렬 없이: df[df.datatype == 'TMAX'].nlargest(10, 'temp_C')
# 정렬 후 인덱스 0,1,2로 다시 매기고 싶으면 ignore_index=True 또는 .reset_index(drop=True)
```

A11.

inplace=True는 원본을 직접 바꾸고 아무것도 돌려주지 않음(None). 그걸 df에 다시 대입했으니 df가 None이 되어 .head()에서 에러. 고치는 법: df = df.rename(columns={...}) 또는 df.rename(columns={...}, inplace=True) 둘 중 하나만.

A12.

```
df['date'] = pd.to_datetime(df.date)          # (a) 문자열 → 날짜 타입
df['temp_C_whole'] = df.temp_C.astype('int')  # (b) 소수 → 정수 (소수점 버림)
df.dtypes                                     # (c) date가 datetime64[ns]로 바뀌었는지 확인
# (d) 날짜 타입이어야 df['2018-10'] 같은 날짜 슬라이싱, 월별 groupby/resample, 날짜 차이 계산이 됨.
# 문자열이면 그냥 글자라서 "10월만" 같은 선택이 불가능.
# ※ df.loc[:, 'date'] = ... 방식은 최신 pandas에서 안 바뀔 수 있음 (수업 때 그대로 object였던 이유)
```

A13.

(a) S&P;는 주말·휴일에 데이터가 없어서 그 날짜엔 비트코인 값만 더해짐 → 인덱스(날짜)가 안 맞아서 생기는 문제.
(b) S&P;의 인덱스를 비트코인 인덱스(매일)에 맞추고(reindex) 빈 날은 앞 값으로 채운(ffill) 뒤 더하기.

```
sp_fixed = sp.reindex(bitcoin.index, method='ffill')  # 주말 행 만들고 금요일 값으로 채움
fixed_portfolio = sp_fixed + bitcoin                 # 이제 날짜가 같으니 그냥 더하면 됨
```

A14.

NaN은 자기 자신과도 같지 않다고 정의되어 있어서(np.nan == np.nan 은 False) == 로는 절대 못 찾음. NaN은 반드시 .isna() 또는 .isnull() 로 찾아야 함.

```
df[df.TMAX.isna()]
```

A15.

(a) inner: 78,780행. weather의 모든 관측소가 station_info에 있으니 weather 쪽은 하나도 안 잃음. (b) right: 78,949행 = 78,780 + 169. 날씨 기록이 없는 관측소 169개가 행으로 추가되고, 그 행들의 weather 쪽 열(date, datatype, value 등)이 NaN. (c) outer: 78,949행 (여기선 right와 같음. weather 쪽에서 잃는 게 없으니까). NaN 위치도 같음.

A16.

```
weather[
    (weather.datatype == 'SNOW')
    & (weather.value > 0)
    & weather.station.str.contains('US1NY')
]
# query 안의 and → 마스크에서는 &, 조건마다 괄호
```

A17.

agg(mean): Busan 20, Seoul 20 → 2행 (그룹 수만큼 줄어듦). transform(mean): 20, 20, 20 → 3행 (원래 행 수 그대로, 각 행에 자기 그룹의 평균을 붙여줌). transform은 "내 값이 우리 그룹 합계의 몇 %인가" 같이 원래 행에 그룹 값을 붙여 계산할 때 씬.

A18.

(1) 'first' (시가 = 그날 첫 거래 가격) (2) 'max' (고가 = 하루 중 최고) (3) 'min' (저가 = 최저) (4) 'last' (종가 = 마지막 가격) (5) 'sum' (거래량은 합계). 가격은 평균 내면 의미가 없고 각각의 정의대로 뭉쳐야 함.

A19.

(a) line — 시간에 따른 변화 (b) scatter — 두 변수의 관계 (점 겹치면 alpha나 hexbin) (c) hist 또는 box — 값의 분포 (d) bar/barh — 범주별 개수 비교 (e) area — 쌓인 전체 높이가 합계, 각 색 두께가 나라별 비중.

A20.

fig = Figure(도화지 한 장), axes = Axes(그래프 칸) 3개가 든 배열. 두 번째 줄은 가운데 칸(axes[1])에 Italy, Spain 선 두 개를 그림. alpha(투명도)는 선이나 점이 겹칠 때 뒤에 있는 것도 보이게 하려고 씬.

5. 시험 직전 10분 체크리스트

아래 문장을 보고 바로 "왜?"가 설명되면 준비 끝. 막히는 게 있으면 해당 노트북의 주석 다시 보기.

- pandas는 위치가 아니라 **인덱스 이름표**가 같은 것끼리 계산한다 → 짝 없으면 NaN
- **loc** = 이름 + 끝 포함 / **iloc** = 번호 + 끝 미포함 / 그냥 **df[a:b]** = 끝 미포함
- 필터링은 **df[(조건) & (조건)]**. and/or 쓰면 에러, 괄호 빼도 에러
- pandas 메서드는 대부분 **원본을 안 바꾼다** → 다시 대입하거나 **inplace=True**. 둘 다 쓰면 None
- **assign** 안의 **lambda x** = 지금 만들어지는 중인 표. 방금 만든 열을 바로 쓸 때 필요
- **wide**(항목이 열로, 보기 편함) ↔ **long**(항목이 행으로, 처리하기 편함). 바꾸기: melt / pivot
- 정렬: **sort_values(by=[...], ascending=[...])**. 큰 것 N개만: **nlargest**. 인덱스 새로: **reset_index**
- 타입 변환: **pd.to_datetime()**(날짜), **astype()**(정수·카테고리). 확인: **dtypes**
- 날짜가 인덱스면 **df['2018-10']**처럼 자를 수 있다 → 그래서 **set_index('date')**
- 인덱스가 안 맞는 두 표를 합치면 이상해진다 → **reindex(다른표.index, method='ffill')**
- NaN은 **isna()**로만 찾을 수 있다 (**== np.nan**은 항상 False)
- **merge**: inner는 겹치는 것만, left/right는 한쪽 전부 유지(짝 없으면 NaN), outer는 전부. 전후 행 수 비교
- **groupby('열').계산()** = 그룹별로 묶어서 계산. agg는 행이 줄고, transform은 행 수 유지
- 그래프: 시간 변화 line / 관계 scatter / 분포 hist·box / 개수 bar. **Figure** 안에 **Axes**, ax=로 칸 지정

※ 최신 pandas에서 교재 코드가 안 돌아가는 부분(append, datetime_is_numeric, loc로 타입 바꾸기 등)은 시험에 "코드 쓰기"로 나올 땐 교재 방식으로 써도 될 가능성이 높지만, 헛갈리면 교수님께 어느 버전 기준인지 물어보는 게 안전해요.